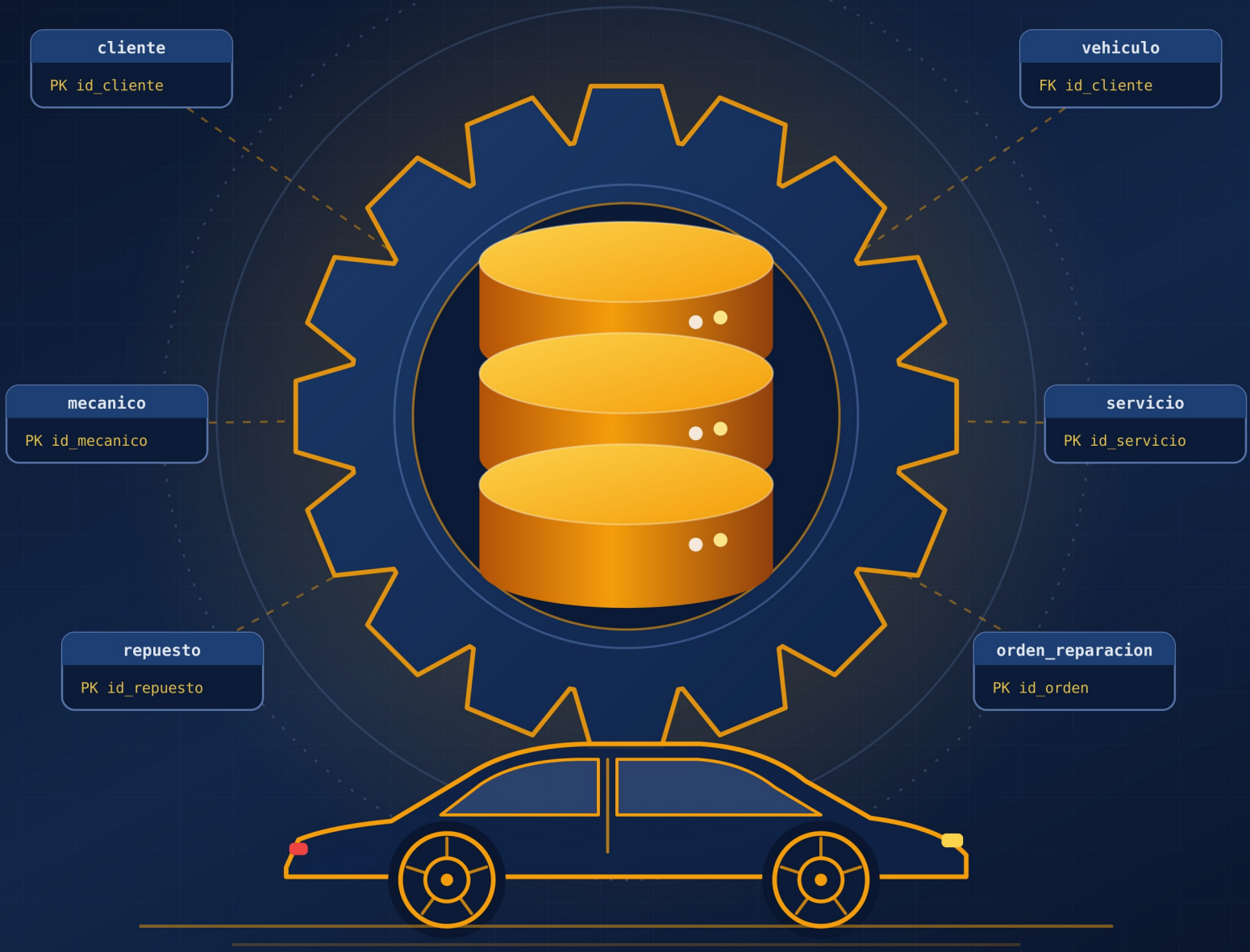
PROYECTO FINAL · BASE DE DATOS 1



Manual técnico

Sistema de gestión de órdenes de reparación

Arquitectura · Conexión MySQL · Despliegue local · Mantenimiento

NOVIEMBRE DE 2026

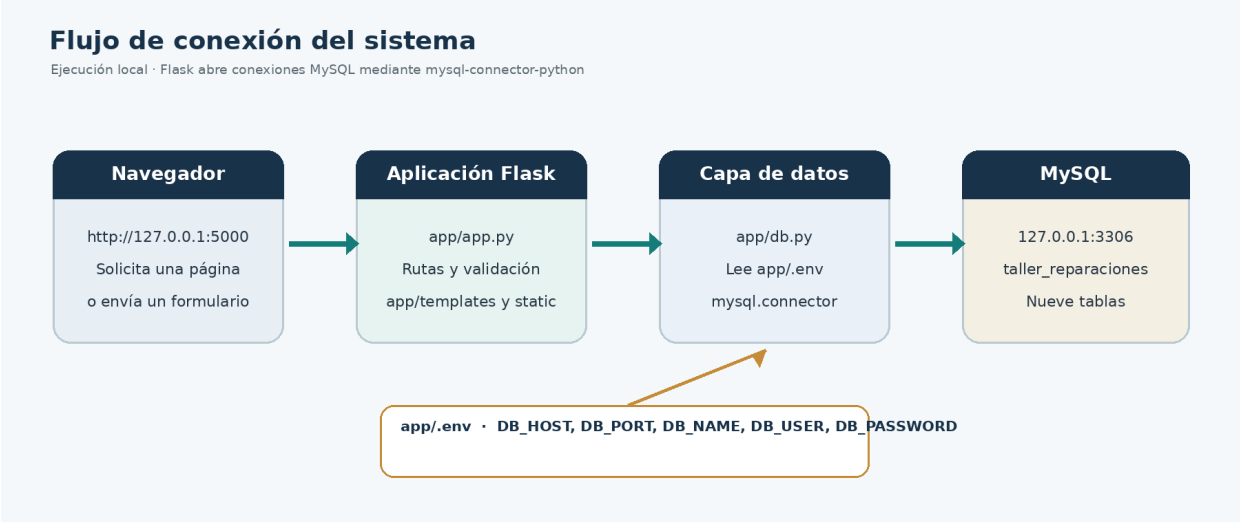
Propósito y alcance

Este manual describe cómo está construido y cómo se prepara el sistema para una ejecución local, está dirigido a quien instala el proyecto, configura MySQL o necesita entender el recorrido de los datos entre el navegador, Flask y la base relacional.

La aplicación permite registrar clientes, vehículos, órdenes, servicios, repuestos y participación de mecánicos, también ejecuta siete reportes SQL, la entrega es académica y local el alcance actual no incluye autenticación de usuarios ni publicación segura en internet.

Arquitectura de la aplicación

El navegador solicita páginas a Flask en el puerto local 5000, las rutas de app/app.py consultan app/db.py para abrir una conexión MySQL las plantillas Jinja construyen el HTML y app/static/styles.css aplica el diseño, el archivo app/.env aporta las credenciales sin incorporarlas al código fuente.



Componente	Responsabilidad
app/app.py	Define las rutas de inicio, clientes, órdenes y reportes; valida formularios y coordina transacciones.
app/db.py	Lee app/.env y construye conexiones con mysql.connector.
app/report_queries.py	Carga los siete SELECT desde sql/03_consultas.sql.
app/templates/ y app/static/	Presentación HTML con Jinja y hoja de estilos CSS.
sql/	Creación del esquema, datos de demostración y consultas del proyecto.

Modelo relacional

El esquema taller_reparaciones organiza las operaciones en nueve tablas, las relaciones uno a muchos se representan con claves foráneas; las relaciones de varios a varios se resuelven mediante tablas de detalle o participación, la figura muestra el modelo entidad-relación entregado con el proyecto.

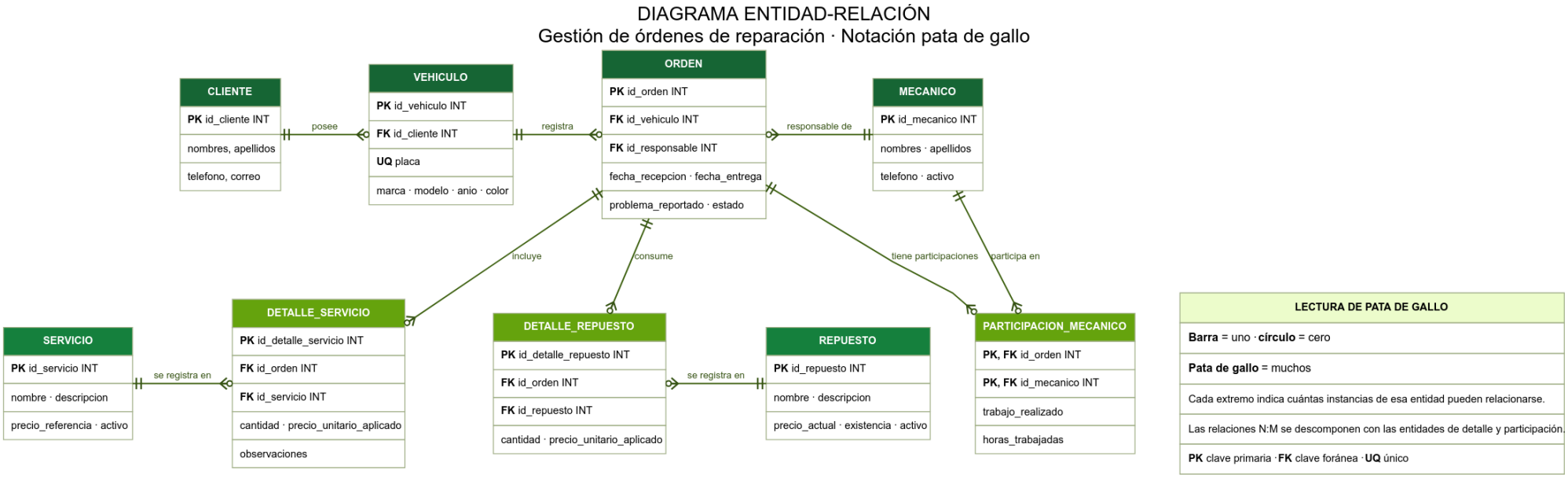


Diagrama entidad-relación del taller, PK identifica clave primaria, FK clave foránea y UQ valor único.

Configuración de la conexión MySQL

La conexión se concentra en connect_db() de app/db.py, al importar ese módulo, _load_env_file() lee app/.env línea por línea, usa os.environ.setdefault(), de modo que una variable definida en el sistema prevalece sobre la del archivo, las variables obligatorias para identificar el servidor y esquema son DB_HOST, DB_NAME y DB_USER.

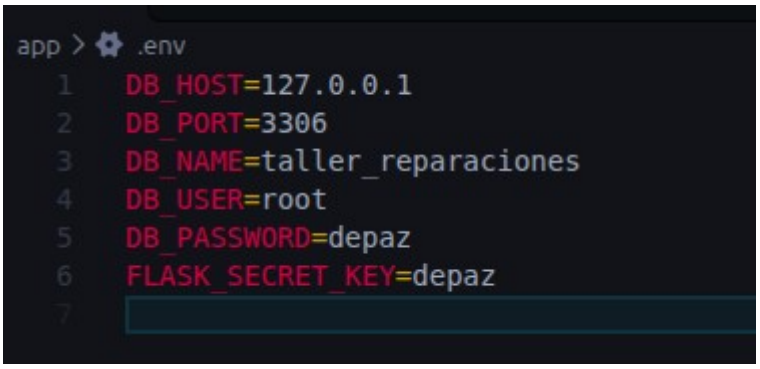
Variable	Uso	Valor de referencia
DB_HOST	Servidor MySQL.	127.0.0.1
DB_PORT	Puerto TCP. Si se omite, usa 3306.	3306
DB_NAME	Esquema de la aplicación.	taller_reparaciones
DB_USER	Cuenta MySQL configurada localmente.	La cuenta que ya utilizas
DB_PASSWORD	Contraseña de esa cuenta.	Se guarda solo en app/.env
FLASK_SECRET_KEY	Clave para firmar sesiones Flask.	Cadena local privada

Preparar credenciales

1. Abra o cree app/.env dentro de la carpeta app.

2. Completa DB_USER y DB_PASSWORD con una cuenta MySQL válida, y confirma DB_HOST, DB_PORT y DB_NAME.

3. Conserva app/.env en el equipo local.



Permisos requeridos en tiempo de ejecución

La aplicación consulta datos y registra filas; además actualiza la existencia de repuestos, la cuenta necesita SELECT, INSERT y UPDATE sobre taller_reparaciones, la creación de tablas corresponde a una cuenta administrativa; no se ejecuta al iniciar “Flask”.

Cómo se abre y cierra la conexión

Cada consulta abre una conexión nueva, ejecuta un cursor y cierra cursor y conexión al terminar. mysql.connector recibe charset utf8mb4, collation utf8mb4_0900_ai_ci y autocommit=False, en la ruta de órdenes, todos los detalles y el descuento de inventario se confirman juntos si ocurre un error, se ejecuta rollback y la transacción no queda parcialmente aplicada.

Esquema, restricciones y reglas de integridad

Tabla	Contenido principal
cliente	Datos de contacto del propietario.
vehiculo	Placa, marca, modelo y propietario.
mecanico	Personal activo y mecánicos responsables.
orden_reparacion	Vehículo, responsable, fechas, problema y estado.
servicio	Catálogo de trabajos y precio de referencia.
detalle_servicio	Servicios por orden, cantidad y precio histórico.
repuesto	Catálogo de partes, precio y existencias.
detalle_repuesto	Consumo de repuestos por orden y precio histórico.
participacion_mecanico	Mecánicos participantes, trabajo y horas.

El script sql/01_crear_bd_y_tablas.sql crea el esquema, sus tablas y cinco índices adicionales. Requiere MySQL 8.0.16 o superior para aplicar las restricciones CHECK y usa la intercalación utf8mb4_0900_ai_ci.

- Claves foráneas mantienen la relación entre órdenes, vehículos, responsables y catálogos.
- RESTRICT protege clientes, vehículos, mecánicos y catálogos referenciados por historial.
- CASCADE se reserva para detalles y participaciones dependientes de una orden.
- Las cantidades deben ser positivas; precios, existencias y horas no pueden ser negativos, la fecha de entrega no puede anteceder a la recepción.
- El precio aplicado se guarda en cada detalle para conservar el costo histórico aunque después cambie el catálogo.

Estados de orden

El esquema MySQL acepta ABIERTA, EN_PROCESO, FINALIZADA y ENTREGADA. app/app.py también incluye Cancelada en la lista del formulario, pero ese valor no está definido en el ENUM de la tabla y puede provocar un rechazo al guardar.

Registro transaccional de una orden

1. Valida vehículo, responsable, fechas, problema y al menos un servicio.

2. Inserta la orden y consulta el precio vigente de cada servicio activo.

3. Bloquea cada repuesto consultado, verifica existencia y calcula el nuevo saldo.

4. Inserta detalles y participaciones; descuenta unidades y hace commit al finalizar.

5. Ante validación fallida o error MySQL, hace rollback y muestra el mensaje en el formulario.

Instalación y puesta en marcha

Requisitos

- Linux Ubuntu o Debian, Python 3.10 o superior y el paquete de venv de la versión de Python instalada.
- MySQL 8.0.16 o superior activo y esquema taller_reparaciones creado.
- Dependencias listadas en app/requirements.txt: Flask y mysql-connector-python.

Preparación inicial de la base

1. Ejecuta sql/01_crear_bd_y_tablas.sql con una cuenta administrativa de MySQL.

2. Ejecuta sql/02_insertar_datos.sql solo si necesitas los datos ficticios de demostración.

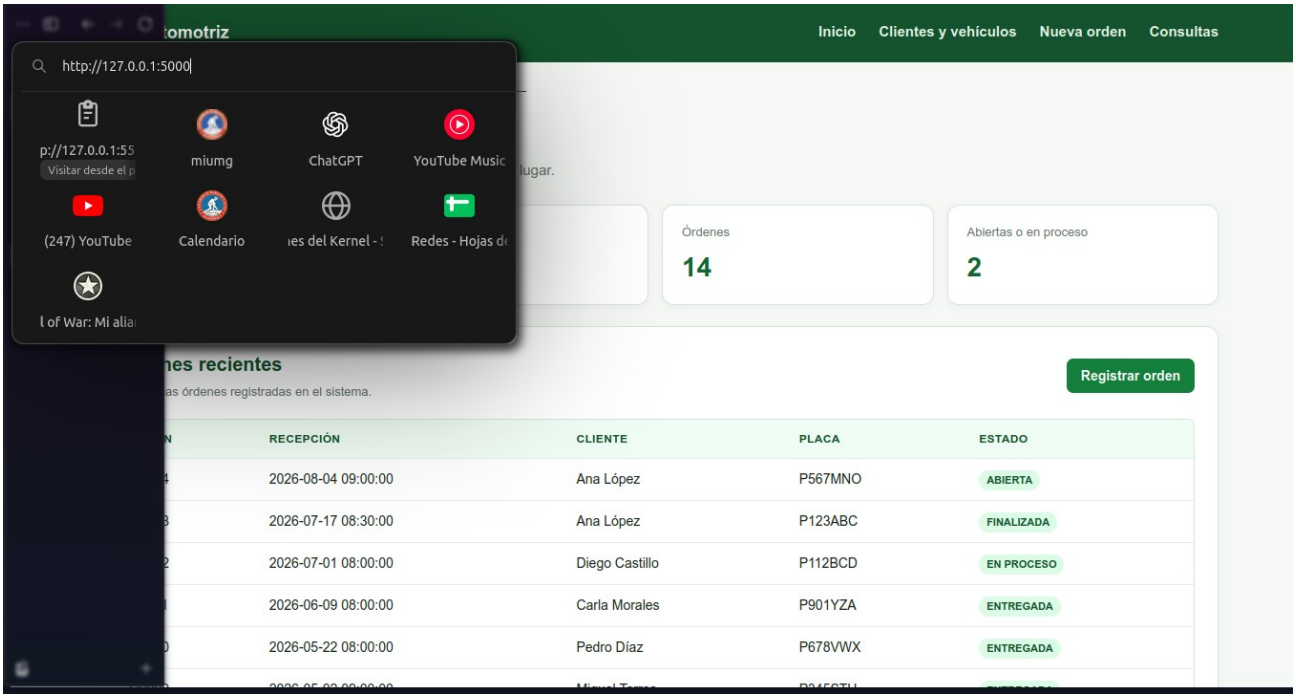
3. Completa app/.env con la cuenta que utilizará la aplicación.

Instalar y ejecutar en Ubuntu 24.04

```
(.venv) yovani@yovani-HP-NoteBook:~/Documentos/Entrega_Proyecto_Final$ python3 -m venv .venv
(.venv) yovani@yovani-HP-NoteBook:~/Documentos/Entrega_Proyecto_Final$ source .venv/bin/activate
(.venv) yovani@yovani-HP-NoteBook:~/Documentos/Entrega_Proyecto_Final$ python -m pip install -r app/requirements.txt
Requirement already satisfied: Flask<4.0,>=3.0 in ./.venv/lib/python3.12/site-packages (from -r app/requirements.txt (line 1)) (3.1.3)
Requirement already satisfied: mysql-connector-python<10.0,>=8.4 in ./.venv/lib/python3.12/site-packages (from -r app/requirements.txt (line 2)) (9.7.0)
Requirement already satisfied: blinker>=1.9.0 in ./.venv/lib/python3.12/site-packages (from Flask<4.0,>=3.0->-r app/requirements.txt (line 1)) (1.9.0)
Requirement already satisfied: click>=8.1.3 in ./.venv/lib/python3.12/site-packages (from Flask<4.0,>=3.0->-r app/requirements.txt (line 1)) (8.5.1)
Requirement already satisfied: itsdangerous>=2.2.0 in ./.venv/lib/python3.12/site-packages (from Flask<4.0,>=3.0->-r app/requirements.txt (line 1)) (2.2.0
)
Requirement already satisfied: jinja2>=3.1.2 in ./.venv/lib/python3.12/site-packages (from Flask<4.0,>=3.0->-r app/requirements.txt (line 1)) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in ./.venv/lib/python3.12/site-packages (from Flask<4.0,>=3.0->-r app/requirements.txt (line 1)) (3.0.4)
Requirement already satisfied: werkzeug>=3.1.0 in ./.venv/lib/python3.12/site-packages (from Flask<4.0,>=3.0->-r app/requirements.txt (line 1)) (3.1.9)
(.venv) yovani@yovani-HP-NoteBook:~/Documentos/Entrega_Proyecto_Final$ python app/app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Acceso y detención

Flask escucha en 127.0.0.1:5000, solo en el equipo local. Abre http://127.0.0.1:5000. Para detenerlo, vuelve a la terminal del proceso y presiona Ctrl+C. MySQL se administra como servicio separado.



Puesta en marcha en Windows

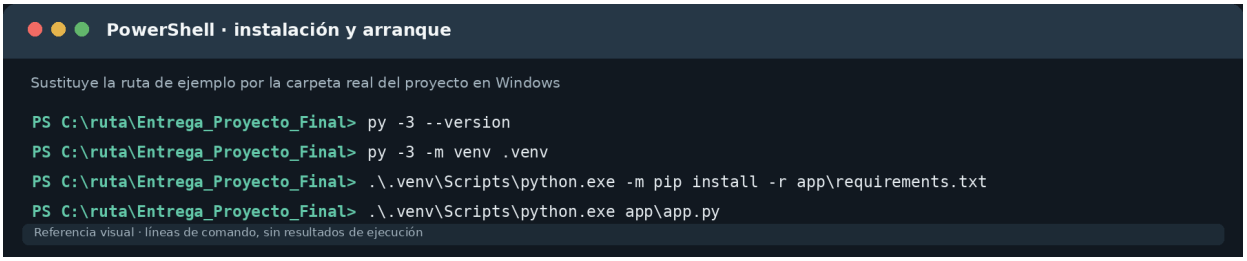
Estas instrucciones aplican si copias el proyecto a un equipo Windows, Python 3.10 o superior debe estar instalado con el lanzador py instalar MySQL Server o confirma que DB_HOST apunte a un servidor MySQL accesible desde ese equipo.

Preparar MySQL y las credenciales

- La base de datos y sus datos no se trasladan automáticamente desde Linux a Windows, usa una instancia MySQL que ya contenga taller_reparaciones y los registros necesarios, o restaura allí una copia de la base antes de arrancar la aplicación.
- Para iniciar MySQL en el mismo equipo, abre servicios desde el menú Inicio, busca MySQL80 (el nombre puede variar) y elige Inicial, si MySQL está en otra computadora, configura DB_HOST con su dirección IP o nombre de red 127.0.0.1 siempre apunta al equipo Windows actual.
- Si el esquema aún no existe, abre MySQL Workbench, conecta al servidor, abre sql/01_crear_bd_y_tablas.sql y ejecútalo, ejecuta sql/02_insertar_datos.sql.
- Crea o edita app\.env con el host, puerto, esquema, usuario y contraseña de MySQL, si MySQL se ejecuta en el mismo Windows, DB_HOST suele ser 127.0.0.1.

Instalar, levantar y acceder

Abre PowerShell en la carpeta raíz del proyecto, si la carpeta está en otra ubicación, usa su ruta real, la imagen contiene los comandos de referencia.



1. Deja abierta la ventana de PowerShell mientras uses el sistema.

2. Abre http://127.0.0.1:5000 en el navegador y registra clientes, vehículos y órdenes, o consulta los reportes.

3. Para detener Flask, vuelve a PowerShell y presiona Ctrl+C, para usarlo de nuevo, repite el comando de arranque de la imagen.

Consultas, mantenimiento y seguridad

app/report_queries.py carga las consultas desde “sql/03_consultas.sql”, el cargador elimina comentarios de línea, separa por punto y coma, ignora USE y exige exactamente siete “SELECT”, al añadir un reporte hay que mantener alineados el SQL, el identificador y el título en “REPORT_TITLES”.

N.º	Reporte
1	Resumen del costo por orden.
2	Historial de reparaciones por vehículo.
3	Mecánicos con más de cinco participaciones.
4	Servicios realizados en más de cinco órdenes.
5	Repuestos con consumo total superior a diez unidades.
6	Ingresos por cliente y servicio superiores a Q2,000.
7	Clientes con gasto por encima del promedio.

Diagnóstico rápido

Síntoma	Revisión
Falla al conectar	Verifica que MySQL esté activo y que DB_HOST: DB_PORT, DB_NAME, DB_USER y DB_PASSWORD correspondan a la cuenta y al esquema correctos.
No existe la tabla	Comprueba si se ejecutó el script de estructura en taller_reparaciones.
Los reportes no devuelven filas	Confirma que hay datos que cumplan los filtros del reporte; por ejemplo, algunos requieren más de cinco órdenes o superar un importe.
Falta un módulo Python	Activar: “.venv” e instala “app/requirements.txt” dentro de ese entorno.
Puerto 5000 ocupado	Detén la instancia Flask previa o cambia PORT mediante una variable de entorno antes del arranque.